
Applying the Blue Angel criteria to Free Software

The handbook

KDE Eco

2022-09-09



Part I: Environmental Impact Of Software

In 2021, the Association of Computing Machinery (ACM), the oldest scientific and educational computing society in the world, released a Technology Policy Council report entitled “Computing And Climate Change.” Among other findings, the techbrief explores the exponential increase in energy and resource consumption of internet-connected devices and artificial intelligence. The report’s estimates are alarming. In 2021 alone, the Information and Communication Technology (ICT) sector is estimated to contribute between 1.8–3.9% of global carbon emissions. To put this into perspective, the aviation industry is estimated to contribute 2.5% of all global emissions. Perhaps most alarmingly, the report warns that if nothing is changed, by 2050 the carbon emissions attributable to the ICT sector will rise to more than 30% of all emissions globally.

In their conclusions, researchers acknowledged an inherent contradiction of digitalization: while on the one hand, digital technology “can help mitigate climate change,” but to do so it “must first cease contributing to it” (p. 1).

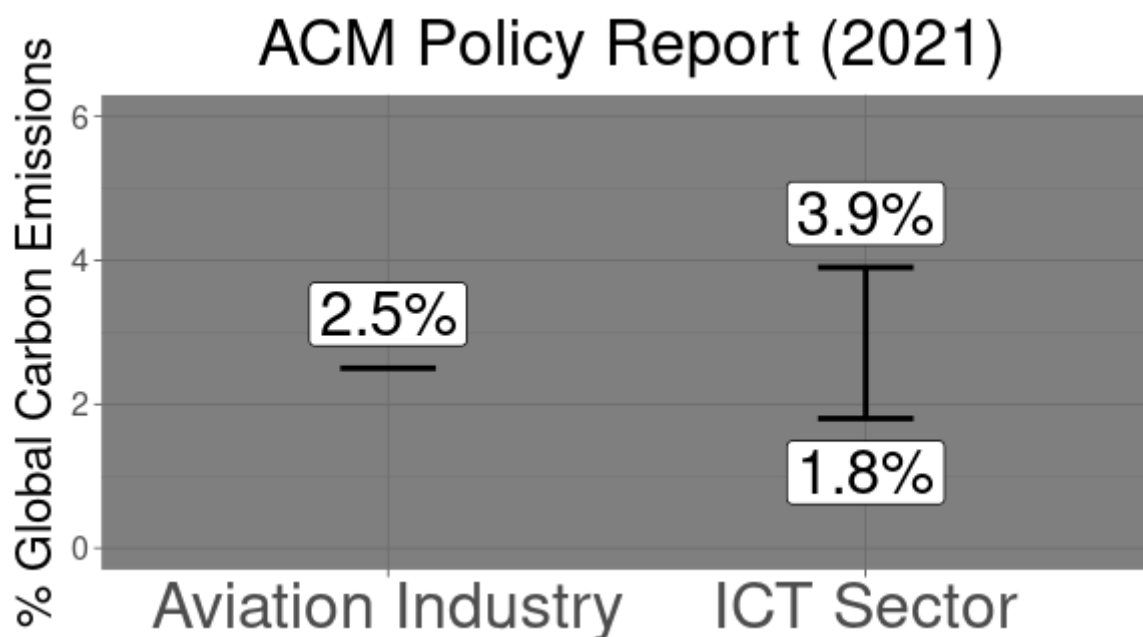


Figure 1: Plot comparing the greenhouse gas emissions of the aviation industry with the ICT sector according to ACM’s 2021 Technology Policy Council report (image published under a CC0 public domain license).

ICT has revolutionized how we live, and it is often praised for bringing convenience and efficiency to our daily lives. Companies have leveraged digital technology for the efficient distribution of all sorts of consumer goods and dematerializing everyday products. Vehicles, including cars, scooters, and

bicycles, are made readily available for rental through apps, eliminating the need for individuals to own them in order to use them. Video streaming means not having to produce and transport DVDs or Blu-ray discs, and driving to the rental store to pick one up on a Saturday night is a thing of the past. E-readers have replaced entire bookshelves. Over the first two decades of the 21st century—with the global SARS-CoV-2 pandemic making digitalization an integral aspect of daily life—video conferences have been able to supplant once in-person activities for just about *everything*, including office meetings, five o'clock drinks, global academic conferences, local piano recitals, and even first dates ... now possible from the comfort of one's home.

For all the ways that technological developments have seemingly made our lives less material and wasteful, and thus more convenient and efficient, it might seem like the rapid pace of digitalization has done more good than harm when it comes to achieving sustainability goals.

But has it really?

The Internet and the devices we use to connect to it require infrastructure—real, physical hardware which demand energy and consume resources. Oft overlooked are the environmental impacts of the factories that produce these devices, the mines for rare earth metals necessary to make them work, and the continent-spanning cables connecting our world. All of this infrastructure requires energy in its day-to-day use. What's more, hardware that is no longer used either ends up in disposal centers for end-of-life treatment (which uses even more energy), or as e-waste that is toxic to people and to the environment. There is a hard material aspect of digitalization.

Within this broader picture, the critical role that *software* plays in contributing to this environmental harm may be overlooked. Indeed, in many cases it is the software that determines both the resources and energy consumed in the operating life of digital infrastructure. As the Blue Angel award criteria for the eco-certification of desktop software states:

A fact that is even more rarely appreciated is that the key to increasing energy efficiency and protecting natural resources lies not with the hardware but rather above all with the software.
(p. 5)

This handbook will take a closer look at some of the ways digital technology is contributing to environmental harm and the climate crisis. To be clear, this handbook is not anti-technology—indeed, digitalization has improved life in countless ways for vast numbers of people. But the ecological impacts of digital technology require us to think more deeply about the ways that we use it and how we might use it more efficiently. And the good news is that through software design, developers can have an immediate, and significant, influence on many of the issues discussed here.

Throughout the text, and especially in the later parts of this text, I will use the Blue Angel ecolabel for desktop software as a benchmark for what a sustainable digital society might look like.

But what is “Blue Angel”, anyway?

The Blue Angel (German: *Blauer Engel*) environmental label is the official environmental label of the German government. In 2020, the German Environment Agency (German: *Umweltbundesamt*, or UBA) released the award criteria for certifying desktop software products. It was the first in the world of environmental certifications to link transparency and user autonomy with sustainability. Free and Open Source Software, or FOSS, has a real advantage here—and by the end of this manual, we hope you’ll have a better understanding why.

But in order to address a problem effectively, we first have to identify what the problem is. So let’s first understand some of the harm that digitalization is doing.

“Material Footprint Of Digital Technology”

Digital technology is often (erroneously) associated with being immaterial. But there is a very real and very material aspect to digitalization: this includes not only physical devices such as smartphones and laptops, but also processing plants for mined metals and ships transporting hardware, as well as cables and data centers connecting the world. The report “Lean ICT: Towards Digital Sobriety” from the SHIFT Project describes this issue succinctly:

[T]he material footprint of digital technology is largely underestimated by its users, given the miniaturization of equipment and the “invisibility” of the infrastructures used. This phenomenon is reinforced by the widespread availability of services on the “Cloud,” which makes the physical reality of uses all the more imperceptible and leads to underestimating the direct environmental impacts of digital technology. (p. 10)

To bring the tangible reality of the “cloud” back down to earth—underwater, even—we need to shift our perspective to the hidden infrastructure that provides the basis for our digital lives. One New York Times article quipped: “[P]eople think that data is in the cloud, but it’s not. It’s in the ocean.” What’s referred to here are the global, underseas communication cables spanning the globe. However, although data transfer itself may be happening underwater, the ICT sector’s carbon emissions are entering the atmosphere at an alarming rate. At COP27 in November 2022, United Nations Secretary General António Guterres underscored the urgency of the moment when he stated: “We are on a highway to climate hell with our foot on the accelerator.” Within the ICT sector, what is contributing to this atmospheric rise of CO₂?

The ACM policy report mentioned in the introduction states that between 2012 and 2018, the energy demands of artificial intelligence (AI) has increased 300,000 times, and currently doubles every few months. To illustrate this, training a single AI model, such as a model used in machine translation and language modeling, required the energy equivalent of flying from New York to San Francisco round-trip ... 300 times (or about 626,000 pounds of CO₂)! In fact, according to the ACM report, data centers currently consume about 3% of the global electricity supply. Another notorious contributor to exploding

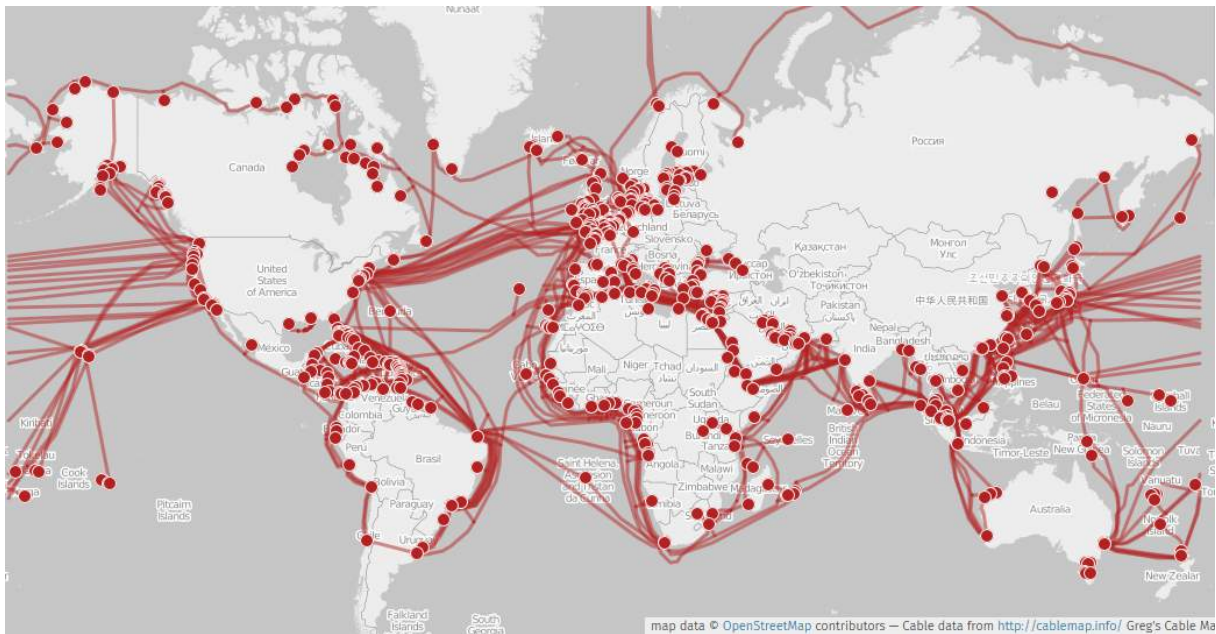


Figure 2: World map of submarine communication cables (cable data by Greg Mahlknecht, KML file released under GPLv3; world map by Openstreetmap contributors).

energy consumption is blockchain technology—in particular, proof of work systems such as Bitcoin, which Harvard Business Review reports requires as much energy as entire countries like Sweden or Malaysia.

Furthermore, the number of internet-connected devices—not only laptops and smartphones, but also smart TVs, home assistants, and other IoT devices—is rapidly growing. By 2025, the number is expected to surpass 75 billion, or just about 10 devices for every person on earth. (The global distribution is far from even, though.) Manufacturing and discarding these devices is causing serious environmental harm, while the production and transportation of devices, as well as their usage, demand energy. Smartphone adoption in particular is increasing rapidly, as are the resource demands to manufacture new, increasingly powerful devices. These are just some of the big contributors to the material footprint of digitalization.

A report from the Shift Project, a French nonprofit aimed at transitioning away from fossil fuels, provides a big-picture view of digital technology’s energy consumption distribution. In “Lean IT—Towards Digital Sobriety” published in 2018, 55% of the energy consumption is attributed to device usage (e.g., personal computers, mobile phones, TVs and home entertainment systems, referred to as “terminals”) along with network traffic and data center computing (*right*), whereas 45% of energy consumption can be attributed to hardware production (*left*).

It is crucial to underscore, however, that energy consumption is not the same as carbon emissions.

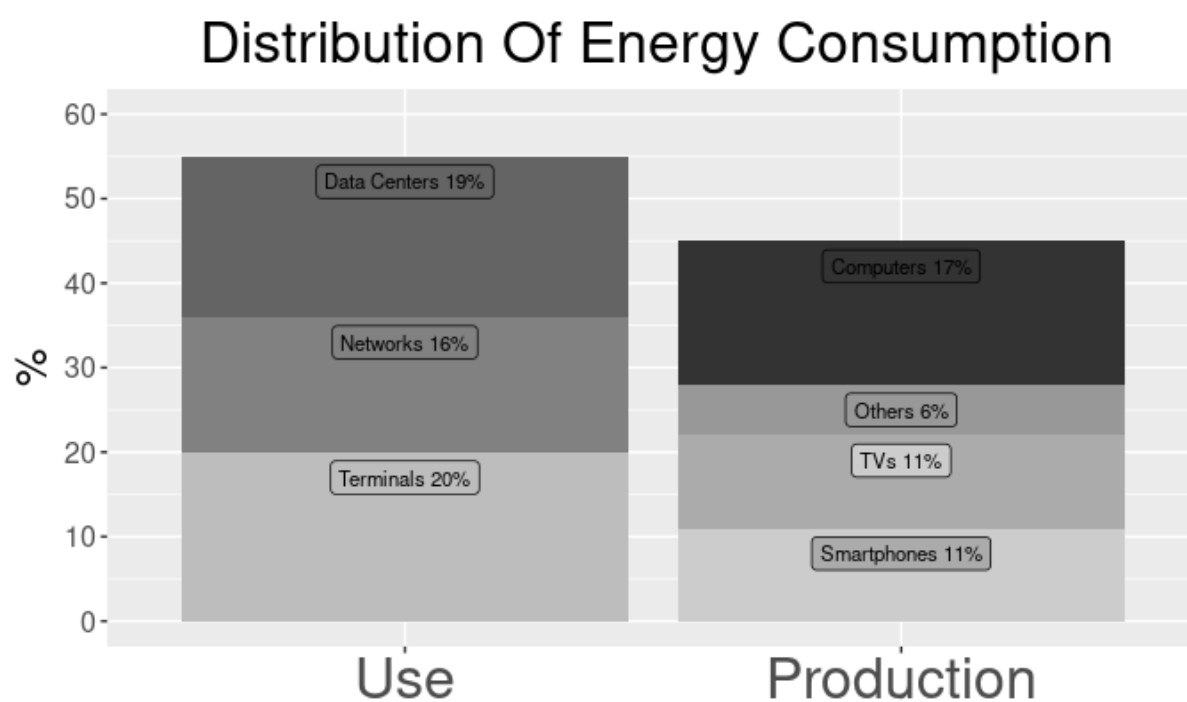


Figure 3: Energy consumption distribution according to the Shift Project report (Image published under a CC0 public domain license).

Carbon emissions depend on the particular mix of fuels used for generating electricity in a particular region, referred to as the electricity or power generation mix. As an example, for the European Union's energy supply in 2016, the power generation mix included 32.9% oil, 23.9% gas, 14.9% coal, 13.7% nuclear power, and 14.5% renewables. With the energy crisis of 2022, the energy mix in the EU has changed—in some cases for the better in the long-term, in some for the worse in the short-term. Relative carbon emissions will depend on this mix: for example, energy consumption from 100% carbon-neutral energy sources contribute no direct CO2 emissions.

In comparison to the comprehensive ACM report, the data in “LEAN IT” is more limited in scope¹—and the data was collected in 2017, before a global pandemic changed many habits in our digital lives. Nevertheless, we'll soon explore how software plays an overlooked but highly significant role in contributing to those numbers ... and perhaps most surprisingly, both in terms of production and usage! First, though, let's go to the place where our digital devices will ultimately land after software has stopped running on them: in the waste bin.

A “Tsunami Of E-Waste”: Dispatch From The Landfill

The Waste Electrical and Electronic Equipment Directive (WEEE Directive) is a European directive on waste electrical and electronic equipment. This Directive sets collection, recycling, and recovery targets for all types of electrical goods. In 2005, two years after the Directive became European law, the Royal Society of Arts in the UK unveiled a sculpture titled “WEEE Man”, designed by Paul Bonomini and fabricated by Stage One Creative Services. Originally located on London's South Bank, the statue is made from 3.3 metric tons of electrical waste—the average amount of e-waste that one UK individual creates in a lifetime. The towering figure was subsequently moved to the Eden Project in Cornwall, where it currently resides.

At seven meter's tall, the “WEEE Man” is a giant. E-waste is considered the “fastest-growing waste stream in the world,” with 44.7 million metric tons generated in 2016—equivalent to 4,500 Eiffel towers, which stacked would be almost 17 times higher than Mount Everest, the tallest mountain in the world. In 2018, an estimated 50 million metric tons of e-waste was reported, motivating the UN to refer to a “tsunami of e-waste rolling out over the world.” The numbers continue to rise: in 2021, an estimated 57 million metric tons of e-waste was generated globally. Less than 20 percent of it is collected and recycled, and although it makes up only 2% of trash in landfills, it contributes to almost 70% of the toxic waste found there.

Electronic scrap components like CPUs contain potentially harmful materials such as lead, cadmium, beryllium, or brominated flame retardants. The end of life treatment of e-waste can also involve significant risk to the health of workers and their communities. What's more, scavengers risk their

¹This section includes material from the Wikipedia article “Software bloat”. The text is released under the Creative Commons Attribution-Share-Alike License 3.0



Figure 4: Photograph of WEEE Man statue by James T.M. Towill (image published under CC BY-SA 2.0 license).



Figure 5: A young man is pictured burning electrical wires to recover copper at Agbogbloshie, Ghana, as another metal scrap worker arrives with more wires to be burned (image by Muntaka Chasant, published under a CC BY-SA 4.0 International license).

health for the discarded precious metals in laptops and smartphones “laced with lead, mercury or other toxic substances”. The process of dismantling and disposing of e-waste has led to a number of environmental impacts in developing countries. Liquid and atmospheric emissions end up in bodies of water, groundwater, soil, and air—and thus, also in land and sea animals, in crops eaten by both animals and humans, and in our drinking water. Environmental pollution is a crucial aspect of digital technology’s environmental harm.

When Less Is Not More: Relative Harm In Production Costs

It’s tempting to assume that with digital technology less is indeed more. Digitalization is often associated with “dematerialization”: printing concert or travel tickets on paper is no longer necessary, as they can be simply be downloaded and presented on one’s smartphone; entire bookshelves can be replaced by a single e-reader; photographs are not collected in over-stuffed shoeboxes, but on a tablet or hard drive; thousands of films and TV series are streamed on laptops, making movie collections a thing of the past. In many cases, one device, a smartphone, is used for all of the above—and much, much more.

All of those material objects which were once a major part of our daily lives—but today, they are simply no longer needed. That must be better for the earth, no?

When estimating the potential ecological benefits of digitalization, however, it is necessary to account for the entire life cycle of an item, including the costs of their production and transportation (to and from the shop, as well as the landfill), or the costs of remediating environmental harm from e-waste. This is especially true when considering digitalization’s carbon footprint, since in most cases the production of devices, together with their transportation and end of life treatment, contribute more greenhouse gas emissions than the devices’ use over their *entire* operating life. To illustrate this, consider Apple’s 2019 Environmental Responsibility Report, which estimates that Apple contributed 25.2 million metric tons of CO₂ in 2018 (p. 9): most of this—eighty percent(!)—comes from production (74%), transportation (5%) and end-of-life treatment (<1%); only 19% comes from the actual usage of devices.

So when are the costs of manufacturing a digital device to replace all of those analogue objects worth it? The book “Smarte Grüne Welt” (English: *Smart Green World*) by Steffen Lange and Tilman Santarius (2018) explores the difficulty of accounting for relative environmental harm when trying to answer such questions. Consider this excerpt, in which the authors explore the environmental impact of printing paper books vs. manufacturing e-readers (pp. 29–31; unofficial translation):

Making electronic devices is obviously more energy-intensive and resource-intensive than printing a single book. For example, the production of an e-reader, usually weighing less than 200 grams, accounts for about 15 kilograms of different materials (especially non-renewable metals and rare

earths), 300 liters of water, and 170 kilograms of the greenhouse gas carbon dioxide. However, it is not just the quantities of input and output materials that are decisive, but also their environmental impact. There are great differences between e-readers and books, especially in the toxicity of materials and manufacturing processes. It is true that the paper industry in many countries (still) has very negative environmental effects, for example when chlorine or acids poison local waters. However, the environmental effects of the electronics industry are sometimes devastating: e-readers and other IT products include brominated fire retardants, phthalates, beryllium, and numerous other chemical substances that are severely harmful to health and the environment. Not to mention the social consequences, such as the sometimes miserable working conditions under which cobalt, palladium, tantalum, and other resources of digital devices are initially extracted in dictatorships such as the Republic of Congo or in other countries of the global South—and then disposed of there at the end of life as environmentally-harmful e-waste.

Despite all of this, the e-reader may be better than the book. This ultimately depends on two factors: How many books are read on the e-reader over its lifetime—and how many people share an analog book? In order for the high environmental costs of the e-reader's production to pay off ecologically, a certain number of books must be read on it. This is the case after 30 to 60 books—depending on the thickness of the book and depending on the environmental indicator. If you read less than this number of books on an e-reader, it is better to choose the paper form. If you go beyond this, each other book on the e-reader is ecologically better than its analog counterpart. Furthermore, how objects are used is crucial [...]: If it is assumed that someone buys a book and does not let anyone else look at it, then a file on the e-reader is around five times more energy-efficient than a book. This advantage, however, disappears when several people share a book.

So, does replacing physical objects with digital technology result in a reduced environmental impact? Well, it depends. For the case of e-readers, will you read 30–60 books before discarding the device? A Gallup poll found that in 2021, 57% of US Americans read fewer than 5 books a year, and 15% read between 6–10 books. This means for over two-thirds of the US population, an e-reader would need to be used for five to ten years in order to be the more environmentally friendly choice. But how many consumers upgrade to the next, shiny new device well before then?

Moreover, will these devices remain supported for up to five years, let alone a decade? Wikipedia's list of discontinued e-readers, as of November 2022, includes seventy-one devices. According to the list, the average lifespan—from intro year to end year—was 1.5 years. How many of those e-readers were still functioning, but ended up in a landfill because of discontinued software support? This kind of premature hardware obsolescence is an important contributor to environmental harm, whether in the form of e-waste or the carbon emissions associated with the device. As Lange and Santarius write, “it is questionable whether all e-readers sold, before they break or become technically obsolete again, are used so intensively on average that an overall ecological benefit is achieved” (p. 31; unofficial

translation).

Look To The Software

There is a crucial relationship between software engineering—sometimes following a logic unrelated to the primary functionality of the software itself—and sustainability issues. Manufacturers regularly encourage consumers to purchase new devices, often unnecessarily; indeed, they may even enforce it through software design. And because of licensing restrictions on software use and certain forms of vendor dependencies, end users can do little about it.

Programmed obsolescence and software lock-out result in discontinued hardware support for largely economic, not technological reasons; abandonware released under a proprietary license can leave users vulnerable to viruses and other malware—and without alternatives to keep their device safely in use. Underlying infrastructure which users are depending on to run an application—such as software license servers used for access control by software vendors—can go offline, sometimes permanently. Feature creep and other forms of software bloat may render less-powerful hardware obsolete, although users never requested those extra functionalities and would remove them if they could. In these cases, due to software-related dependencies and usage restrictions, still-functioning devices are tossed in the garbage heap while resources are consumed to produce and transport new ones.

Indeed, software bloat is one of several issues the Blue Angel ecolabel identifies as being problematic. Bloat is described in the award criteria for desktop software like so (p. 5):

Processing power has doubled about every two years since 1970. This means that functions are processed twice as fast and thus less energy is required for the same functions. A similar improvement in efficiency cannot be observed in the field of software. [...] The availability of more and more powerful hardware has resulted in software becoming more and more bloated from version to version so that more resources are required **for only minimal or even no enhancement of the functionality**. [emphasis added]

Besides rendering functioning devices unusable, software bloat also results in higher energy consumption when using the software. For example, one study published by the German Environment Agency found that two applications which achieve the same result can have drastically different energy profiles.

In the report Word Processor 1 is identified as Open Source and Word Processor 2 as a proprietary software product. It is important to underscore that both computer programs were running the same sequence of commands through a Standard Usage Scenario (SUS) script. These scripts describe “the most representative use of the respective software over a defined period of time” (p. 23). We’ll return to usage scenarios in Part II of this handbook. For now, what’s important to note is the massive difference

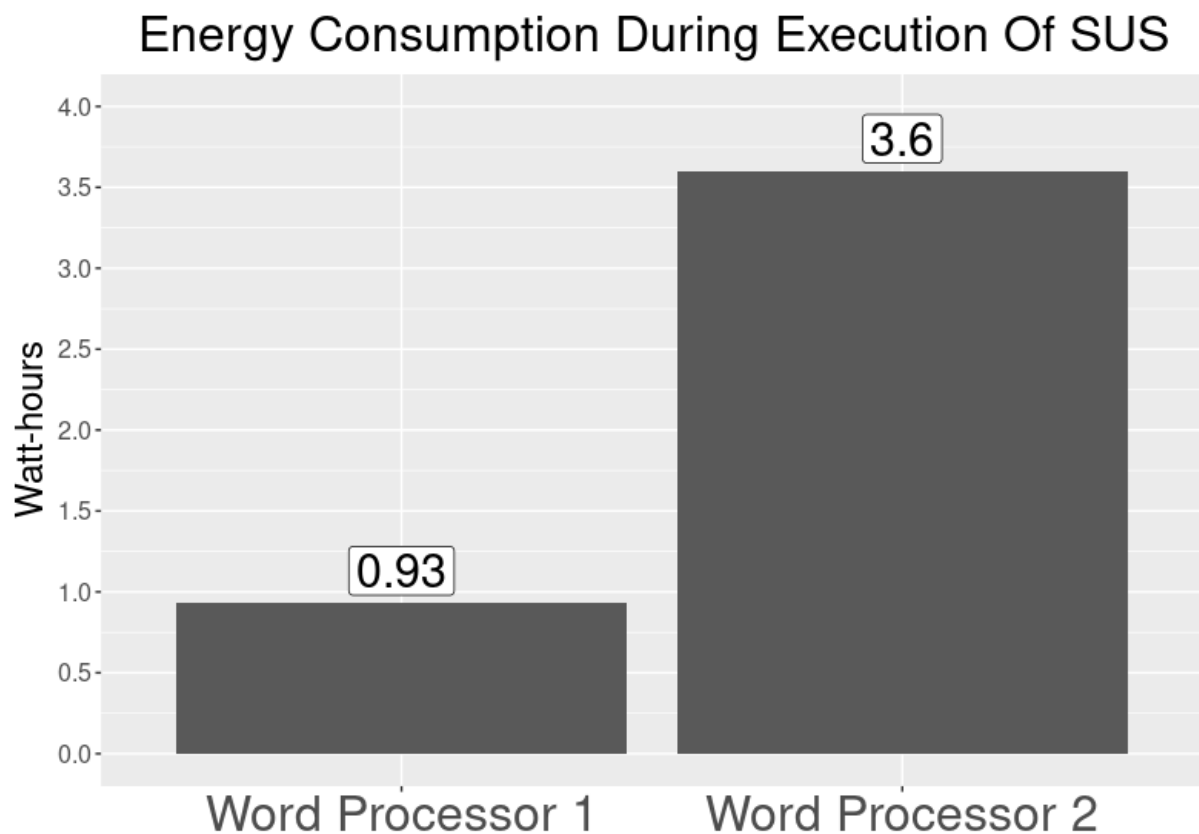


Figure 6: Bar plot comparing two word processors during execution of a Standard Usage Scenario (based on Figure 1 on page 24 in the report). Word Processor 1 is an Open Source program. This word processor consumes four times *less* energy than Word Processor 2, a proprietary program. (Image published under a CC0 public domain license.)

in energy use: running Word Processor 2 requires 4 times the energy consumption of Word Processor 1—again, and this cannot be stressed enough, to achieve the same result!

Further, when looking at the energy consumption of the two word processors over time, it becomes clear how the two software products behave very differently ... and perhaps contrary to expectations. Consider the energy consumption in the plot below, in which the energy consumed over time is shown while running the usage scenario's sequence of commands. At the 440-millisecond mark, the script calls for both word processors to save the document and then stops calling for further action. As you can see, Word Processor 1 goes idle (as one might expect); by contrast, Word Processor 2 continues working, consuming energy even though the script has ended.

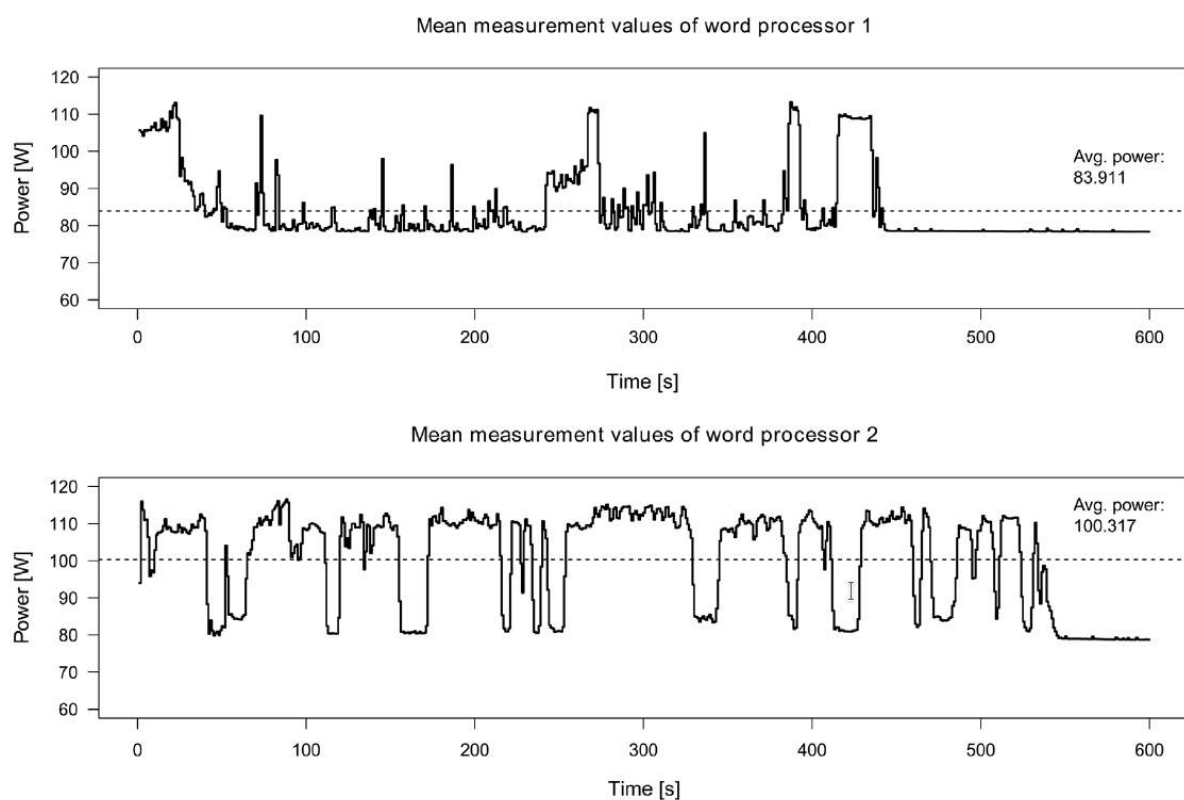


Figure 7: Plot comparing two word processors over time when running a Standard Usage Scenario (SUS). Word processor 1 (*top*) goes into an idle state when not doing anything, seen most clearly after the document is saved at the 430-milliseconds mark and no other actions are emulated by the SUS script. By comparison, word processor 2 (*bottom*) rarely goes idle. (Screenshot from article published under CC BY-NC-ND license.)

It's worth asking what the additional energy consumption from 440–600 milliseconds is for: Are the actions of Word Processor 2 necessary for the functionality of the software? Is the word processor collecting and transmitting user data? If yes, do users have a way to opt out of these types of analytics?

Indeed, user autonomy in turning off of unwanted data use can make a big difference on the energy profile of a software product. Data mining, third-party tracking, personalized engagement-maximizing algorithms, and advertising are significant drivers of energy consumption. Collecting and analyzing user data and training algorithms on it also requires computing power and infrastructure!

In 2021, researchers in the EU estimated the environmental costs of tracking which users cannot opt out of, what they refer to as “unwanted data use” (“Carbon footprint of unwanted data-use by smartphones: An analysis for the EU”). The carbon footprint of this tracking and advertisements by smartphones—between 3 and 8 million metric tons a year in the EU alone—is “equal to the carbon footprint of between 370 and 950 thousand EU citizens” (p. 12). This is roughly the annual footprint of a city like Turin, Italy or Lisbon, Portugal. The report also points out that about 60% of European smartphone users indicate they would opt out of tracking and block advertisements when possible. That’s an awful lot of energy use for something most users don’t want in the first place!

In many ways, digital sustainability is a question of software. Software determines how long devices remain in use as well as the energy consumed during use. It is for this reason that the Blue Angel award criteria look to the characteristics of software design which have a clear impact on health and the environment, which we will return to shortly. Beforehand, though, we will consider two side topics relevant to the discussion here.

“The Very Contrary Is The Truth”: Jevons’ Paradox

The “rebound effect” (also known as the “take-back effect”) describes how efficiency gains can decrease or even negate efficiency benefits. Imagine that a change in software results in a 5% improvement in energy efficiency; however, because of the increased energy savings, you end up using the software more. But as a result of increased usage, you now have less savings.

For the sake of concreteness, imagine that—because you are using the software more—instead of 5% drop there is only a 1% drop in overall energy consumption. The rebound effect is 80% $((5-1)/5)$: in other words, those original efficiency gains have decreased by 80%, practically negating any savings from the improved efficiency! If the rebound effect goes over 100%, meaning more energy is now used than before, this is referred to as Jevons’ Paradox, or “back-fire.” The paradox comes from the English economist William Stanley Jevons, who in 1865 recognized that technological improvements in coal-use actually increased coal consumption across industries. Jevons concluded that:

“It is a confusion of ideas to suppose that the **economical use of fuel is equivalent to diminished consumption**. The very contrary is the truth.” [emphasis added]

A practical interpretation of this paradox is that efficiency gains must be combined with conservation practices in order to have a meaningful effect, lest one ends up consuming more energy than before.

That ACM report—the same one referenced at the beginning of this handbook—makes a similar point: “Computing-enabled efficiencies must be coupled with slashed energy demand to reduce ICT sector carbon emissions.” (p. 1)

Is This Worth It?

Looking at the bigger picture, software’s contribution to environmental harm and global greenhouse gas emissions may be less significant in comparison to other industries. To this point, let us consider four counter-arguments. The first is rejecting the “not as bad as” fallacy, also known as the “Appeal to Worse Problems”. The argument can be summed so: software’s contributions to global CO2 emissions may not be as bad as another industry’s, and therefore it is not worth focusing on. What is wrong with this argument is, even if another industry may be worse, that does not negate the fact that software is responsible for causing serious environmental harm. What’s more, we can reduce software’s environmental impact AND reduce another industry’s at the same time: the “not as bad as” fallacy suggests a false choice.

Let’s not be like XKCD’s White Hat and appeal to worse problems as an excuse not to do anything:

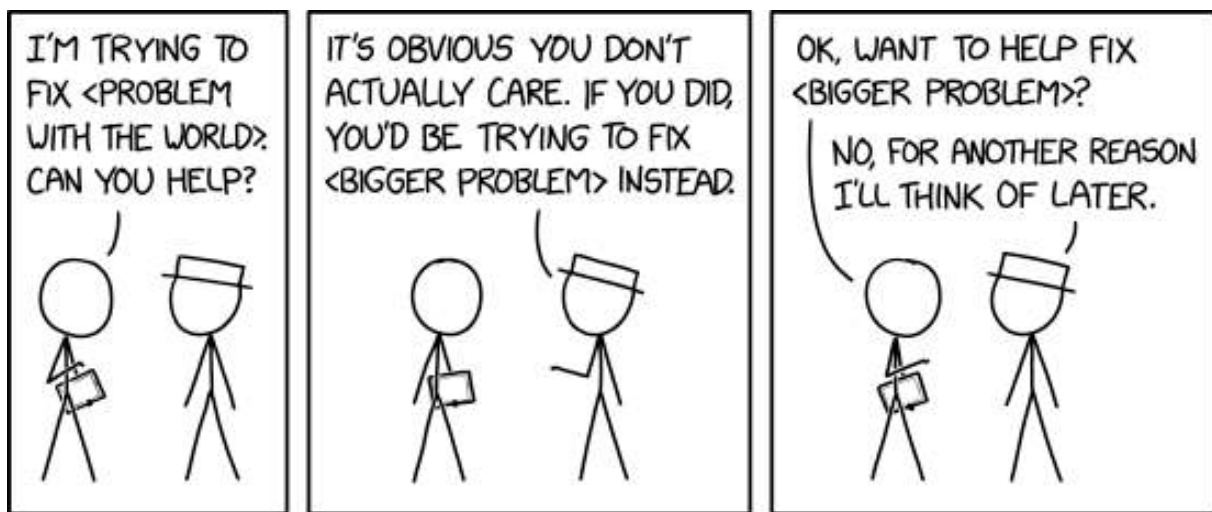


Figure 8: XKCD comic “2368: Bigger Problem” (comic released under CC BY-NC 2.5 license).

Second, focusing only on fixing the “biggest problem” is not necessarily the most effective strategy—it’s also important to weigh the likelihood of success when addressing a problem, as well as the time and resources required to do so. Free & Open Source Software, with its focus on user autonomy and transparency, provides unique opportunities for users, communities, and organizations to directly address intertwined social and ecological issues. FOSS can be adapted, updated, and maintained at lower cost and without vendor dependencies or artificial restrictions.

Third, it's hopefully clear by now that software *does* have significant impacts on energy consumption and the production of waste, both of which have consequences for the environment. When considered at scale, changes in software design can have savings comparable to the annual energy consumption of entire cities. For instance, watch Detlef Thoms (04:20–06:10 do back-of-the-envelope calculations to go from one CPU-second reduction, equivalent to about a 10 watt-second savings, to 95,000 megawatt hour savings simply by scaling up—this is roughly the annual energy consumption of over 30,000 two-person households. As he states in the video, “often, it is a quite manageable set of decisions which lead to significant differences in power consumption”.

Finally, in order to make claims about relative harm, it is necessary to first have estimates about actual effects. Since research in the area of software's energy and resource consumption is still quite new, we often do not have data to make data-driven claims. With this handbook, and with the Blue Angel ecolabel as a guide, KDE hopes to help change that.

Changing our software may seem like a small gesture in addressing as complex as climate change. It's also clear that simply changing our individual consumption patterns may not be sufficient on its own (what's worse, evidence suggests that major contributors to global greenhouse gas emissions—such as ExxonMobile—have embraced a rhetoric of individual responsibility in order to deflect from their own role in the crisis).

It's true: a zero-emissions future will require fundamental shifts in how we live, and that responsibility can't be managed at an individual level. But consider what anthropologist Margaret Mead once observed:

“Never doubt that a small group of thoughtful, committed citizens can change the world; indeed, it's the only thing that ever has.”

Structural change happens when dedicated, passionate people organize to confront pressing societal issues. With decades of experience successfully bringing global communities together to work toward common goals, Free & Open Source Software is very well positioned. We know how to organize—now, it's a matter of doing it. Let's unite to combat software-driven environmental harm. Let's foster a culture of digital sustainability in our software communities. Let's build energy and resource efficient software together!

Part I Sources

Some material in this section is based directly on text from two Wikipedia articles: (i) “Waste Electrical and Electronic Equipment Directive” and (ii) “Electronic waste.” Both texts are released under the Creative Commons Attribution-Share-Alike License 3.0.

PART II: Eco-Certifying Desktop Software

What do construction products, toilet paper, and software have in common?

All can be eco-certified by the Blue Angel environmental label, the official environmental label of the German government!

The Blue Angel ecolabel is awarded to a range of products and services, from paper products and construction materials to printers. In 2020 the German Environment Agency extended the award criteria to include software products, which was the first in the world of environmental certifications to link transparency and user autonomy with sustainability.

In order to obtain the Blue Angel seal, a software product must demonstrate that it meets a list of stringent requirements considered critical for the environment over the product's life cycle. In part this means providing transparency in the energy consumption when using the software, and ensuring the software runs on hardware at least five years old. Moreover, the criteria also include a list of user autonomy requirements which reduce the environmental impact of software.

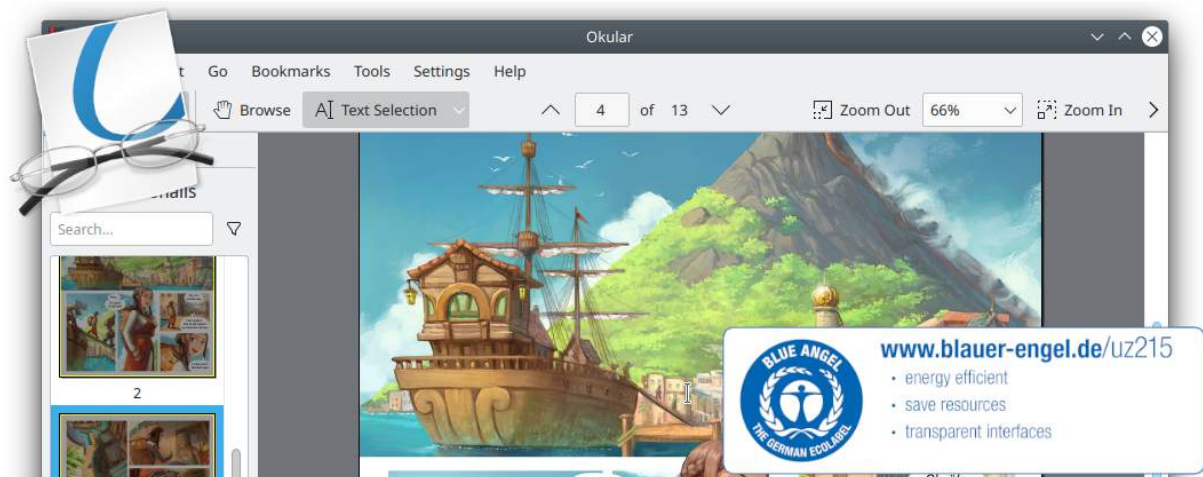


Figure 9: Okular awarded the Blue Angel ecolabel

Okular, KDE's popular multi-platform PDF reader and universal document viewer, was awarded the Blue Angel ecolabel in March 2022. (Image published under a CC-BY-4.0 license)

In this section I will provide a broad overview of the Blue Angel and the ABCs of the award criteria for desktop software, leaving details about compliance for eco-certification to Part III: "How To Measure The Energy Consumption Of Software And Document Backward Compatibility & User Autonomy". Part II will conclude with a presentation of Okular, the first computer program worldwide to receive the label, and an introduction to KDE and the KDE Eco initiative.

The Blue Angel

A Type I Environment Label

The Blue Angel is the official environmental label awarded by the German government. Introduced in 1978, the Blue Angel is the oldest ecolabel in the world. The owner of the label is Germany's Federal Ministry for the Environment, Nature Conservation, Nuclear Safety, and Consumer Protection (German: *Bundesministerium für Umwelt, Naturschutz, nukleare Sicherheit und Verbraucherschutz*, or BMUV). Furthermore, the Blue Angel ecolabel is a member of the Global Ecolabelling Network (GEN), an international network of Type I ecolabels which at the time of writing has 37 members across nearly 60 countries!



Figure 10: Logo of the Blue Angel ecolabel

Logo of the Blue Angel ecolabel. The logo is intentionally designed to correspond to the logo of the United Nations Environment Programme. (Image published under a CC-BY-SA-4.0 license)

What is a Type I environmental label, you may be wondering. For Type I labels the entire life cycle of the product is taken into account, and compliance with the award criteria is assessed by a third-party. (Compliance with Type II environmental labels, by comparison, are self-declared and do not require any third-party auditing.) RAL gGmbH is the third-party non-profit that assesses compliance with the award criteria. RAL is also responsible for awarding the ecolabel.

Product Certification

The Blue Angel ecolabel is currently awarded to around 100 product groups and services across the following sectors: paper products, building products, furnishing, clothing, washing and cleaning agents, cleaning services, household chemicals, packaging, vehicles, energy and heating, household electrical devices, among others. That list now also includes desktop software.

The award criteria for certification are developed transparently by the German Environment Agency, and the process includes the Environmental Label Jury, a body made up of suppliers as well as civil society organizations and research institutions. Importantly, the Blue Angel does not certify that a product is completely harmless. Instead, certified products represent a “lesser evil” with respect to environmental harm. This can be summed up with the motto ‘*as little as possible, as much as necessary*’. Nevertheless, the Blue Angel does not directly compare products — rather, receiving the Blue Angel ecolabel indicates that a product fulfills a list of stringent award criteria for a specific category, which we will explore in detail for software products in this handbook.

Logo

The Blue Angel logo is intentionally designed to correspond to the logo of the United Nations Environment Programme (UNEP), but with the colors reversed. The outer ring contains the words “BLUE ANGEL” (German: *Blauer Engel*) and “THE GERMAN ECOLABEL” (German: *Das Umweltzeichen*). According to the official website, this reflects the aim of the German government to embed the UNEP goals in Germany.²

The ABCs Of The Award Criteria For Desktop Software

The Blue Angel’s award criteria for “Resource and Energy-Efficient Software Products” were released in January 2020. Although not the first Type I ecolabel for software — the Hong Kong Green Council, also a member of the Global Ecolabelling Network, released criteria in 2010 for Green IT software — the

²It is also possible to have a setup using 3 computers, with the Standard Usage Scenario emulation generated on a computer independent of the SUT; see Kern et al. (2018). Details of a similar setup for an external workload generator can be found at the FEEP repository.

Blue Angel criteria are the first both to identify a process for measuring software's energy consumption and to specify ways enabling user choice can reduce environmental harm.

In order to obtain the ecolabel, a software product must demonstrate that it meets a list of stringent requirements considered critical for the environment over the product's life cycle. There are two primary objectives of the Blue Angel for software: (i) to award software with lower performance requirements such that "longer operating lives for [...] hardware are possible"; and (ii) to recognize products which "stand out due to their high level of transparency and give users greater freedom in their use of the software" (p. 6). To achieve this, there are three main categories of the Blue Angel for desktop software, which will be referred to here as the ABCs of the award criteria.

- (A) Resource & Energy Efficiency
- (B) Potential Hardware Operating Life
- (C) User Autonomy

The criteria listed in category (A) require the energy consumption of a software product to be measured and reported; what is more, from the time of certification the energy consumption must not increase by more than 10%. For category (A), it is necessary to obtain energy consumption data using an external power meter as well as hardware performance information such as CPU usage, network traffic, etc., while running the software in a representative way.

The criteria in category (B) ensure the software has low-enough performance requirements to run on older, less powerful hardware. Compliance entails a declaration of backward compatibility, with details about the hardware and the required software stack at least 5 years old on which the software runs.

Finally, the criteria in category (C) ensure that users have an influence on the energy consumption of the software they use. There are eight categories for the autonomy criteria:

1. Data Formats — *Interoperability Gives Users The Choice To Use More Efficient Software*

Data formats should not be used by vendors to lock in users to a specific computer program or create switching costs which are too onerous. Interoperable data formats prevent users from being stuck using a program that consumes a high amount of energy, such as Word Processor 2 from Part I, when a more efficient one can achieve the same results, such as Word Processor 1.

2. Transparency — *Being Open Removes Dependencies, Essential For Long-Term Use*

Transparency must be provided so as not to create software dependencies on any company or organization or create unnecessary restrictions of software use. When security updates are discontinued, the source code should be made available so users and other third parties can continue to maintain the software. Moreover, enhancing the functionality of software must not

be limited by restrictive or undocumented application interfaces (APIs). In fact, issues related to application interfaces are so critical they even came up before the United States Supreme Court in 2021, where there was a favorable ruling for fair use of transparent interfaces.

3. Continuity Of Support — *Security Updates Prevent E-Waste*

Dependency on suppliers for essential updates should not leave users with abandonware, where the software, and thus hardware, cannot be used without serious disadvantages. Thus, security updates should be provided for up to five years after discontinued development or end of sale. Moreover, security updates should be separable from functional updates so users are not coerced into adopting programs suffering from feature creep and other forms of software bloat.

4. Uninstallability — *Removing Unwanted Software Increases Efficiency*

Being able to completely uninstall software that is not needed has ecological benefits, since unwanted software or software components can create inefficiencies by occupying memory, wasting processing time, adding disk usage, consuming storage, and causing delays at system startup and shutdown. When a user no longer wishes to continue using a software product, it should be possible to completely purge it from the system, while keeping all user-generated data.

5. Offline Capability — *Removing Access Control Keeps Software Usable, Decreases Energy Consumption*

Use of the software should be possible without an internet connection — unless of course a network connection is necessary for the software's intended functionality. License servers and other forms of access control restrict use of an application in ways unnecessary to the software's intended functionality. When a server goes down or there is Internet outage, such access control locks people out from using their software, possibly permanently. Moreover, such dependencies use network traffic and thus consume energy in ways unnecessary for the intended functionality of the software.

6. Modularity — *Having Essential Functions Only Decreases Memory And Energy Demands*

Users should be able to install only what they need. Non-essential functions can increase memory and energy demands, making the software less efficient and perhaps unable to run on older hardware. People should have the ability to limit the range of software functions to those that they either want or require, no more and no less.

7. Freedom From Advertising — *Opting-Out Reduces Energy Consumption*

As discussed in Part I, unwanted data use in the European Union alone is roughly equivalent to the annual energy consumption of a city like Lisbon or Turin. Allowing users to opt out of advertizing reduces resource and energy demands on users' devices as well as remotely in the

servers pushing ads, as well as decreases data volume and thus energy consumption in network traffic.

8. Documentation — *For Long-Term, Resource-Conserving Use Of Software, And Therefore Hardware*

Documentation is a prerequisite for long-term viability of a software product. Documentation is also necessary for the resource-conserving use of an application. By documenting the above criteria, users can continue using the software and thus hardware in a sustainable way and developers can continue to maintain the software without dependencies on vendors.

Software products that comply with the award criteria are less likely to suffer from bloat and other inefficiencies. This mitigates problems related to software-driven hardware obsolescence and can reduce e-waste, which in turn results in fewer devices needing to be produced and shipped, and valuable metals do not need to be mined and processed. By ensuring user autonomy as defined above, users directly influence the environmental impact when running software, such as keeping devices in use for longer and reducing the resource and energy consumption while used.

The Blue Angel award criteria for software, with its focus on transparency in resource and energy-efficiency, hardware operating life, and user autonomy, provide an excellent benchmark to begin a discussion on software sustainability and push development in this area forward. Moreover, in FOSS communities we often take user autonomy and transparency and their benefits for granted. Although being Free and Open Source Software is not a requirement to obtain the Blue Angel ecolabel, it is in this category that FOSS really shines — we are already at the forefront of sustainable software design.³

First Eco-Certified Computer Program: KDE’s Popular Document Reader Okular

Okular, KDE’s popular multi-platform PDF reader and universal document viewer, was the first software product to be officially recognized for sustainable software design as reflected in the Blue Angel award criteria for software eco-certification. Introduced in 1978, the Blue Angel is the world’s earliest-established environmental label. In February 2022, Okular was the first-ever software product to be certified with its seal! What is more, Okular is the first eco-certified computer program within the Global Ecolabelling Network representing nearly 60 countries.

In 2021 KDE started KDE Eco, a project with the goal of putting KDE and Free Software at the forefront of sustainable software design. Sustainability is not a new goal for Free and Open Source Software (FOSS). The four freedoms have always put Free Software at the forefront of sustainable software design. What is new is that FOSS values are now recognized as being directly related to sustainability by an organization like the German Environment Agency.

³This section includes material from the Wikipedia article “Software bloat”. The text is released under the Creative Commons Attribution-Share-Alike License 3.0



Figure 11: KDE Eco logo

Logo of the KDE Eco initiative (Image published under a CC-BY-4.0 license)

The Blue Angel award criteria reflect KDE's values and those of the larger FOSS movement seamlessly. With Free and Open Source Software, transparency is guaranteed and control is handed over to users, instead of being held back by vendors or service providers. This allows users to decide what they want from the software they use and, all too often overlooked, the hardware as well. For instance, users may drive down the energy consumption of their programs with no loss in functionality as they can install only what they need, no more and no less; and avoid advertising and data-mining options which run processes in the background consuming resources. As for FOSS developers, they typically continue to support hardware that the industry would be eager to make obsolete, providing users with up-to-date and secure software for devices that might otherwise be discarded as e-waste and end up polluting landfills.

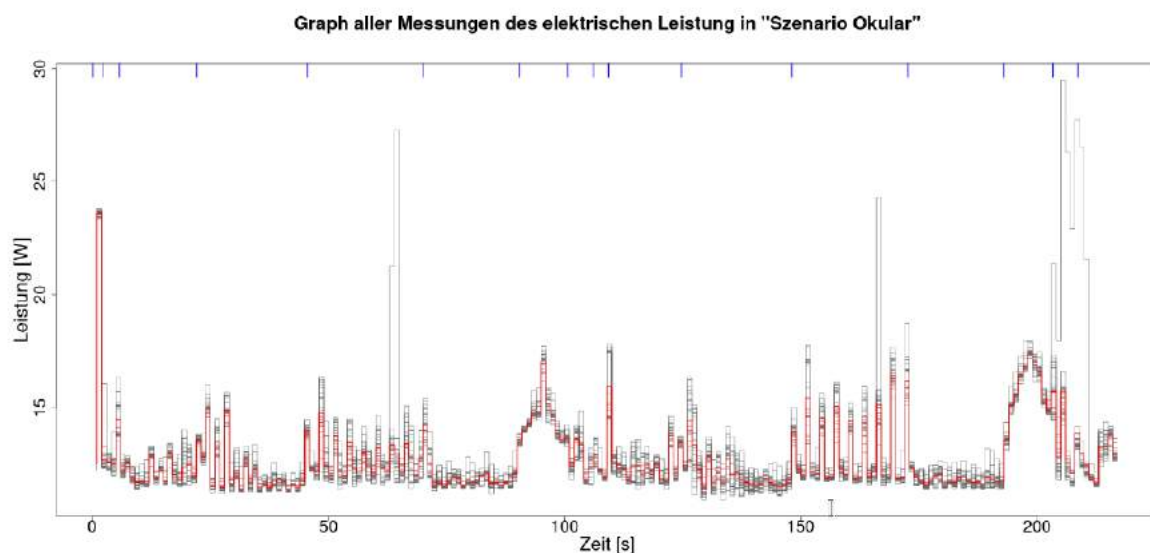


Figure 12: Okular's energy consumption report

Okular's energy consumption report (Image published under a ??? license)

In short, as a consequence of the user freedoms and transparency guaranteed by a Free Software license, users and communities are able to influence the factors determining the resource consumption of their software! Released under the GPLv2+ license, Okular is Free and Open Source Software, and so was also already fulfilling many of the user autonomy criteria necessary to obtain the Blue Angel seal of approval. Further work was carried out to make Okular fully compliant with all of the Blue Angel criteria, and become officially recognized as providing transparency in energy and resource consumption, extending the potential hardware operating life of devices, and enabling user autonomy.

KDE is a world-wide community of software engineers, artists, writers, translators, and creators who are committed to Free Software development. With KDE's long-standing mission and guiding vision, as

well as the talent and capabilities of its community members, it is not surprising that KDE is a pioneer among the group of people and organizations in driving a discussion on sustainable software. Now, with the first ever eco-certified software product, the KDE community is celebrating the achievement together with the wider Free Software community, as well as with the computer science department at Umwelt Campus Birkenfeld, where researchers measured the resource and energy-consumption of Okular and other KDE software.



Figure 13: Okular icon

Icon for KDE's popular application Okular (Image published under a ??? license)

KDE was founded in 1996 and maintains numerous Free and Open Source Software products, including the Plasma desktop environment; the design app for painters and graphic artists, Krita; the GCompris suite of educational activities for children; Kdenlive, a professional video-editing software product; and, of course, Okular, with which you can view all sorts of documents, including PDFs, comics, scientific and academic papers, and technical drawings. Okular also lets you check digital signatures and sign documents yourself, as well as include annotated text and comments directly embedded into the document.

Okular works on Linux, Windows, Android, and Plasma Mobile, and it is available to download for all GNU/Linux distributions, as a standalone package from Flathub and the Snap Store, through the KDE F-Droid release repository for Android, as well as from the Microsoft Store. Being FOSS, the source code is also readily available at Okular's GitLab repository for all to use, study, share, improve, and most of all, enjoy.

KDE and the Free Software community would like to send a heartfelt thank you to the Okular developers for making environmentally-friendly software for all of us!⁴

⁴It is also possible to have a setup using 3 computers, with the Standard Usage Scenario emulation generated on a computer independent of the SUT; see Kern et al. (2018). Details of a similar setup for an external workload generator can be found at the FEEP repository.

Part III: Compliance With The Blue Angel Award Criteria

As described in “PART II: Eco-Certifying Desktop Software”, there are three main categories of the Blue Angel award criteria for desktop software:

- (A) Resource & Energy Efficiency
- (B) Potential Hardware Operating Life
- (C) User Autonomy

In this section I will cover the three categories in depth, providing a hands-on guide to demonstrating compliance with each.

(A) How To Measure Your Software

The lab setup consists of a power meter, a computer to aggregate and evaluate the power meter output, and a desktop computer for the system under test where user behavior is emulated. The setup described here follows the specifications from the Blue Angel Basic Award Criteria for Resource and Energy-Efficient Software Products. Terminology is in part from Kern et al. (2018): “Sustainable software products — Towards assessment criteria for resource and energy efficiency”.

Overview Of Lab Setup

The laboratory setup requires 1 power meter and at least 2 computers:

- *Power Meter*

One of the devices recommended by the Blue Angel is the Gude Expert Power Control 1202 Series (manual). It provides plugs for powering the computer and measures the current during operation. The device can be controlled and read via cabled Ethernet. There is a web-based user interface, a REST API, and the device supports various protocols such as SNMP or syslog.

- *Computer 1: Data Aggregator & Evaluator*

The computer for collecting and evaluating results from the power meter.

A Python script to read out the data from the Gude Expert Power Control 1202 Series is available at the FEEP repository.

It is recommended to monitor progress live with the second computer in order to ensure everything is proceeding smoothly.

Other PMs may require non-Free software, e.g., Janitza’s GridVis Power Grid Monitoring Software.



Figure 14: Image of Gude Power Meter

Gude Power Meter (Image published under a ??? license)

- *Computer 2: System Under Test*

The reference system is the hardware used to measure the energy consumption of the system under test, or SUT. The SUT includes the operating system and software installed for (i) testing the software product, (ii) emulating the standard usage scenario⁵ and (iii) collecting the hardware performance results.

Note the following:

For GNU/Linux systems, the Blauer Engel criteria (Section 1.1) recommend Fujitsu computers as the reference system.

For emulating activity in the standard usage scenario, Free Software task automation tools such as [xdotool](#), [KDE Eco Tester](#) (in progress), or [Actiona](#) (GPLv3) can be used.

For collecting hardware performance data (e.g., processor and RAM utilization, hard disk activity, network traffic), a Free Software tool such as [Collectl](#) (GPLv2/Artistic License) can be used.

It is possible to repurpose cheap switchable power plugs as measurement devices; see Section “Alternative: Gosund SP111 Setup” for set up instructions.

⁵It is also possible to have a setup using 3 computers, with the Standard Usage Scenario emulation generated on a computer independent of the SUT; see Kern et al. (2018). Details of a similar setup for an external workload generator can be found at the FEEP repository.

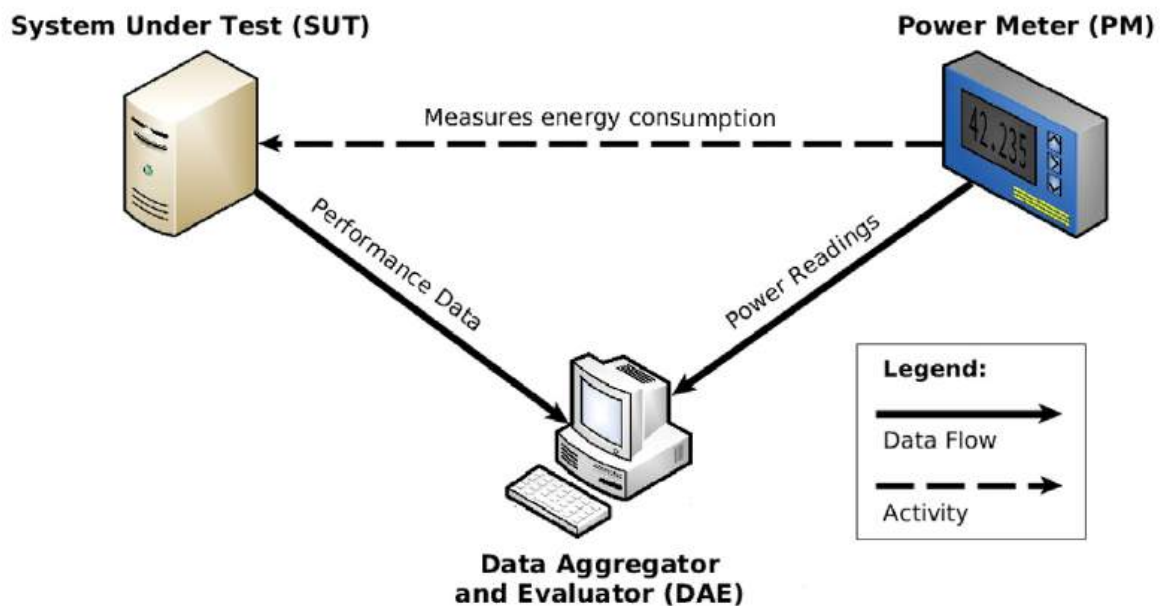


Figure 15: Lab Setup

Gude Power Meter (Image published under a ??? license)

System Under Test (SUT) The Fujitsu Esprimo P920 Desktop-PC proGreen selection (Intel Core i5-4570 3,6GHz, 4GB RAM, 500GB HDD) is one of the recommended reference systems.

On the reference system you can set up the System Under Test (SUT). The SUT must reduce unrelated energy consumption and have a standardized configuration. This includes the following:

- Overwriting the entire hard drive of the machine with a standardized OS.
- Deactivating all possible background processes (automatic updates, backups, indexing, etc.).
- Installing the necessary software (i.e., the application under consideration as well as the emulation and data collection software).

Standard Usage Scenario (SUS) Preparing the SUS requires the following:

- Identifying tasks users typically carry out when using the application under consideration.
- Identifying functionalities which require high energy demand or high resource utilization.
- Based on the above, scheduling a flow chart of individual actions and emulating these actions with a task automation tool.
- Note that when running the SUS the cache should be cleared between runs and any new files deleted before starting the next measurement.

An example standard usage scenario for KMail includes: searching for an email, writing a reply or forwarding the email, saving an attachment, deleting a folder in the mail client, etc. See the Actiona scripts used to test KMail, Krita, and Okular for examples.

Note: If the task automation tool uses pixel coordinates to store the position of the automated clicks (e.g., [Actiona](#)) and, moreover, the screen resolution of the computer used in preparation differs from that of the laboratory computer, all pixel coordinates will eventually have to be set anew for the laboratory environment (Seiwert & Zaczek 2021: p. 12).

Emulation Tools For Usage Scenarios An automation tool is required which can run the usage scenarios and does not need human intervention, so it can be run repeatedly in a well-defined way to provide accurate measurements.

Beyond [xdotool](#), [KDE Eco Tester](#) (in progress), or [Actiona](#), there are other candidates for tools which might meet the requirements. See a list from KDE Contributor David Hurka in the presentation “Visual Workflow Automation Tools”. Most of these tools use X11-specific features and thus do not work on Wayland systems. There are a few possible approaches here:

- Selenium Webdriver using AT-SPI
- The XDG RemoteDesktop portal
- Various Wayland protocols (support varies between compositors):
 - <https://github.com/swaywm/wlr-protocols/blob/master/unstable/wlr-virtual-pointer-unstable-v1.xml>
 - https://api.kde.org/frameworks/kwayland/html/classKWayland_1_1Client_1_1FakeInput.html
- libinput user devices

Measuring Software

The measurement process is defined in Appendix A of the Basic Award Criteria. It requires one to record energy data and performance indicators with a granularity of 1 second and log it so it can be processed and average values can be calculated.

Some general comments:

- Times between the PM and Computer 1 (Data Aggregator & Evaluator) must be synchronized.
- On Computer 1 confirm that the desired power outlet is read out (e.g., via the live graph created when using [LabPlot](#)).
- When using [Collectl](#) to collect performance load, ensure it is running in the console of the SUT; furthermore, check that the required csv file is also correctly generated.

- Since each run of the usage scenarios results in changes to the standard operating system, clearing the cache between runs is recommended.
- All runs (Baseline, Idle Mode, Standard Usage Scenario) must be for the same length of time, based on the time needed to run the usage scenario script.

During the energy measurement we also need to record a set of performance indicators: processor utilisation, RAM utilisation, hard disk activity and network traffic. Tool candidate:

- `Collectl`

The following command is used for obtaining hardware performance data with `Collectl` (Mai 2021: p. 15):

```
$ collectl -s cdmn -i1 -P --sep 59 -f ~/performanceMeasures.csv
```

The specified options are:

- `-s cdmn`
collect CDU, Disk, memory, and network data
- `-i1`
sampling interval of 1 second
- `-P`
output in plot format (separated data which consists of a header with one line per sampling interval)
- `--sep 59`
semicolon separator for -P option
- `-f /PATH/TO/FILE.csv`
save file at specified path

Baseline, Idle Mode, And Standard Usage Scenario

- **Baseline: *Operating System (OS)***
To establish the baseline energy consumption and hardware performance data of the system under test, a scenario is measured in which the OS is booted but no action is taken.
- **Idle Mode: *OS + Application While Idle***
To establish the energy consumption and hardware performance data of the application while idle, a scenario is measured in which the application under consideration is opened but no action is taken.

Important: the baseline and idle mode are run for the same time needed to carry out the standard usage scenario. Since the power consumption for the baseline and idle scenario is relatively uniform, 10 repetitions for each is considered sufficient to obtain a representative sample (Seiwert & Zaczek 2021).

- Standard Usage Scenario: *OS + Application In Use*

To measure the energy consumption and hardware performance data of the application under consideration in use, the standard usage scenario is run. The measurement of the standard usage scenario should be repeated 30 times, taking several hours. This higher number of repetitions is necessary to obtain a representative sample, as the energy consumption and performance data may vary across measurements (Seiwert and Zaczek 2021).

Analysis Of The Results With OSCAR

There is a tool available from Umwelt Campus Birkenfeld which generates reports from measurement data, called OSCAR (Open Source Software Consumption Analysis):

- Source Code
- Running instance.

At the project website there is the Oscar Manual with detailed instructions including screenshots on how to use OSCAR.

CSV Files The analysis with OSCAR requires uploading the following files to the OSCAR website: (i) a log file of actions taken, (ii) the energy consumption data, and (iii) the hardware performance data. All files are .csv files, examples of which are provided below. Important: some preprocessing of the raw data may be necessary (e.g., performance data measured by Collectl; see below for details).

- Log File Of Actions

An example log file of actions for measuring Kate is below. Here iteration number, timestamp in date-time format, and actions are listed in three columns:

iteration 1	2022-05-21 18:54:36	start iteration
iteration 1	2022-05-21 18:55:41	go to line 100
iteration 1	2022-05-21 18:55:46	toggle comment
iteration 1	2022-05-21 18:55:50	find kconfig

iteration 1	2022-05-21 18:55:55	move between searches 6 times
iteration 1	2022-05-21 18:56:05	close find bar
iteration 1	2022-05-21 18:56:05	standby 30 sec
iteration 1	2022-05-21 18:56:35	go to line 200
iteration 1	2022-05-21 18:56:40	select 10 lines
iteration 1	2022-05-21 18:56:43	delete selected text

- Energy Consumption Data

The energy consumption data will have the following format, in which (i) the timestamp increases in one-second increments and (ii) *value* stands for ‘averaged measured value per second in watts’:

```
timestamp; value1
timestamp; value2
timestamp; value3
timestamp; value4
```

Below is an example of the raw output for 7 rows from the Gude Power Meter looks like so. The first column shows the timestamp in nanoseconds in Epoch time, and the second column Watts consumed.

```
1661611923019071 43
1661611923142924 43
1661611924293989 29
1661611924417017 28
1661611924744885 28
1661611924869051 28
1661611924992392 28
```

The raw data can be processed in R: nanoseconds in Epoch time are converted to human-readable date-time forms with the command `as.POSIXct(<NANOSECONDS>/1000000, origin = '1970-01-01', tz = 'Europe/Berlin')`, and Watts are averaged per second. The same data

above is shown below after preprocessing, with n indicating the number of values used to calculate the average:

datetime	n	watts
2022-08-27 16:52:03	2	43.00000
2022-08-27 16:52:04	5	28.20000

Below is another example from the OSCAR Manual. Here measurements are from the Janitza UMG 604 power meter exported on the DAE using the GridVis interface. The columns above containing the min and max values are ignored by OSCAR.

Row-Number;	Time;	Valueavg[W];	Value-min[W];	Value-max[W];
1;	18.01.12 17:00:01;	78,573;	78,281;	78,982;
2;	18.01.12 17:00:02;	78,611;	78,390;	78,972;
3;	18.01.12 17:00:03;	78,556;	78,353;	78,887;
4;	18.01.12 17:00:04;	78,589;	78,391;	78,882;
5;	18.01.12 17:00:05;	78,546;	78,288;	79,057;
6;	18.01.12 17:00:06;	78,580;	78,252;	78,941;
7;	18.01.12 17:00:07;	80,712;	78,330;	87,391;
8;	18.01.12 17:00:08;	78,545;	78,275;	79,202;

- Performance Data (Raw)

When using [Collectl](#) for collecting hardware performance data it is necessary to do the following before uploading the data to OSCAR (see Seiwert & Zaczek 2021: p. 13 for details; see also Appendix A 2 on p. 46 for a Python script to automate some of these tasks):

- All information above the header row can be removed.
- Remove all # characters from the file.
- In the first column no separator value should come between the data-time, otherwise the date and time will be interpreted as two separate columns.
- Moreover, the date should have a character inserted between YYYYMMDD, e.g., YYYY-MM-DD as above. Whatever character is used must be specified in OSCAR.
- The file must be saved in csv format.

Moreover, the hardware performance output from [Collectl](#) includes many columns not necessary for analyzing the data with OSCAR. The only measurements that need to be specified in OSCAR are the following columns: [CPU]Totl = Processor; [MEM]Used = Main memory - used kilobytes; [NET]RxKBTot = Network - Kilobytes received/s; [NET]TxKBTot = Network - Kilobytes sent/s; [DSK]ReadKBTot = Disk - Kilobytes read/s; and [DSK]WriteKBTot = Disk - kilobytes written/s.

Below is an example of the preprocessed results from [Collectl](#) measuring the performance data for Kate. The timestamp again increases in one-second increments.

Date_Time	[CPU]Totl%	[MEM]Used	[NET]RxKBTot	[NET]TxKBTot	[DSK]ReadKBTot	[DSK]WriteKBTot
2022-08-27 16:47:10	1	7131968	0	0	0	0
2022-08-27 16:47:11	4	7131968	0	0	0	0
2022-08-27 16:47:12	1	7131968	0	0	0	0
2022-08-27 16:47:13	1	7131968	0	0	0	120
2022-08-27 16:47:14	1	7131968	0	0	0	0
2022-08-27 16:47:15	1	7131968	0	0	0	56
2022-08-27 16:47:16	1	7131968	0	0	0	48
2022-08-27 16:47:17	1	7131968	0	0	0	0
2022-08-27 16:47:18	1	7131968	0	0	0	0
2022-08-27 16:47:19	4	7131968	0	0	0	132

Uploading Data Once the necessary files have been preprocessed, the evaluation of the baseline, idle mode, and standard usage scenario can begin. Note that the baseline measurements are uploaded separately from the idle mode/standard usage scenario measurements.

When uploading the idle mode measurements, for *Art der Messung* ('Type of Measurement') select *Leerlauf* ('Idle Mode'); when uploading the standard usage scenario measurements select *Nutzungsszenario* ('Usage Scenario').

For each measurement, the log file of actions taken, energy consumption, and hardware performance data are uploaded.

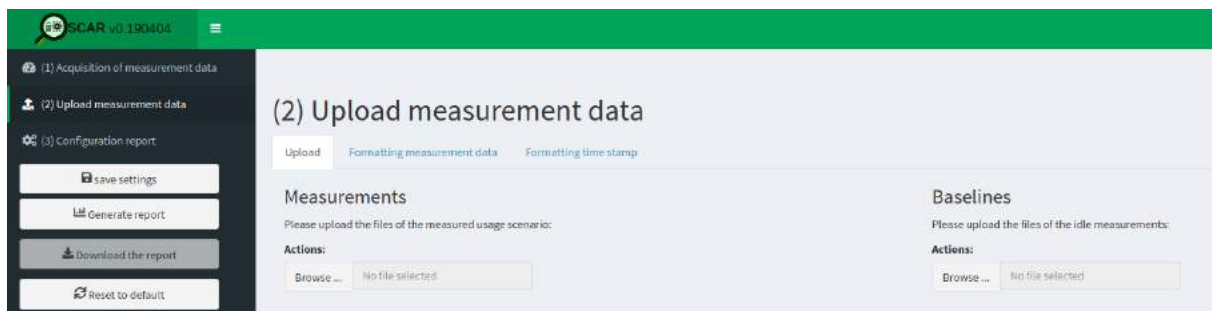


Figure 16: Screenshot of the OSCAR website

Screenshot of the OSCAR website.

In the OSCAR interface, note the following:

- The duration of the individual measurements (for instance, the duration of a measurement in seconds) must be specified.
- The formatting of the data must be specified.
- Only the semicolon must be specified as separator.
- For the performance output of the hardware, it is necessary to select the columns that are relevant for evaluation.
- As a last step, the correct formatting of the time stamp must be specified for each of the uploaded files, e.g., %Y-%m-%d %H:%M:%OS.

Downloading Reports After completing the above, the report can be generated and downloaded, resulting in two documents: one is a report for the baseline and idle mode, and the other is a report for the baseline and standard usage scenario.

Submitting Documentation For Blue Angel

For Blue Angel eco-certification, it is necessary to complete several documents with the above information as well as the energy consumption reports. The information that needs to be included is as follows:

- Details about the software (name, version, etc) and measurement (when and where measurements were made, etc.)
 - Technical details about the power meter (instrument, sampling frequency, length of scenario, sample size)
 - Technical details about the reference system (year, model, processor, cores, etc.)
 - Software stack used for measurements (`xdotool`, `Collectl`, etc.)
 - Minimum system requirements (processor architecture, local working memory, etc.)
 - Energy consumption measurements are found in the OSCAR reports or equivalent.
 - Hardware utilization measurements, which includes the following (for IDLE use Idle Mode measurements, and for SUS use SUS measurements):
 - *Full Load*: “For processing power, the full load is 100%, for working memory the sum of the installed RAM capacities, for network bandwidth the maximum transmission speed, etc.” (p. 23)
 - *Base Load*: Average load for the reference system in Baseline measurements
 - *Idle/SUS Load*: Average load for the reference system for IDLE/SUS measurements
- From the above measurements, the following calculations are made for hardware utilization (for IDLE use Idle Mode measurements, and for SUS use SUS measurements):
- * *Net Load*: $\text{IDLE/SUS Load} - \text{Base Load}$
 - * *Allocation Factor*: $\text{Net Load} / (\text{Full Load} - \text{Base Load})$
 - * *Effective Load*: $\text{Net Load} + \text{Allocation Factor} * \text{Base Load}$
 - * *Hardware Utilization* (SUS only): $\text{Effective Load} * \text{Time (seconds)}$

More References

The measurement process is also described in the following documents:

- Resource and Energy-Efficient Software Products: Basic Award Criteria (Edition January 2020, Version 1)
- Mai, Franziska (2021) *Vergleichende Analyse und Bewertung von Betriebssystemen hinsichtlich ihrer Energieeffizienz* (German only),
- Seiwert & Zaczek (2021) *Projektbericht: Ressourceneffiziente Softwaresysteme am Beispiel von KDE-Software* (German only),
- OSCAR Manual (German only), and

- Section 4.1 of Kern et al. (2018), *Sustainable software products—Towards assessment criteria for resource and energy efficiency*.

Alternative: Gosund SP111 Setup

Thank you to Volker Krause for his work on hacking inexpensive power plugs to use as power meters and documenting the process in details. You can read more at the following blog posts from Volker's blog:

- "Cheap Electric Power Measurement"
- "KDE Eco Sprint July 2022"

Below is a guide to setting up a Gosund SP111 power plug already flashed with Tasmota firmware in 10 or fewer steps.

Although the data from this inexpensive power meter likely will not be accepted by the Blue Angel for eco-certification, it is nonetheless possible to obtain preliminary data using this tool.

- (0) *Prerequisite*

It is necessary to have the power plug already flashed with a sufficiently new Tasmota version.

- (1) *Firmware Reset*

If the device had previously been connected to another Wifi it might need a full reset before being able to connect to a new one.

If the device did open a WiFi access point named "tasmota-XXXXX" this is not needed, continue at (2) directly.

Press the button for 40 seconds.

The device will restart and you should be able to continue at (2).

- (2) *WiFi Setup*

The device opens a WiFi access point named "tasmota-XXXXX", connect to that.

Open `http://192.168.4.1` in a browser.

The device asks you for the WiFi name and password to connect to after entering those. The device will reconnect to that WiFi and disable its access point.

While doing that it should show you its new address in the browser — make a note of that.

In case that did not happen, check your WiFi router for the address of the device.

- (3) *Tasmota Setup*

Open the address from step (2) in a browser.

You should see the Tasmota web UI (a big “ON/OFF” text and a bunch of blue and one red button).

Click “Configuration”.

Click “Configure Other”.

Copy

```
1 {"NAME":"Gosund SP111 2","GPIO":  
2 [56,0,57,0,132,134,0,0,131,17,0,21,0],"FLAG":0,  
3 "BASE":18}
```

into the template input field.

Tick the “Activate” checkbox.

Click “Save”.

The device will restart, connect to it again.

The UI should now also contain text fields showing electrical properties, and the “Toggle” button should now actually work.

- (4) *Calibration*

Open the address from step (2) in a browser.

Connect a purely resistive load with a known wattage, such as a conventional light bulb (not a LED or energy-saving bulb).

Switch on power by clicking “Toggle” if needed.

Verify that the “Power Factor” value is shown as 1 (or very close to 1); if it is lower the current load is not suited for calibration.

Click “Console”.

Enter the following commands one at a time and press enter:

```
1 AmpRes 3  
2 VoltRes 3  
3 EnergyRes 3  
4 WattRes 3  
5 FreqRes 3  
6 SetOption21 1  
7 VoltageSet 230
```

Enter the command `PowerSet XXX` with XXX replaced by the wattage specified for the test load (e.g., “40” for a 40W light bulb).

Click “Main Menu”.

The main page now should show correct power readings with several decimals precision.

- (5) *MQTT Broker Setup*

The only known way for high-frequency automatic readouts so far is polling over MQTT. This is not ideal and needs additional setup, unfortunately.

If you happen to have a MQTT Broker around already, skip to step (6); otherwise you need to set up one. The below assumes Mosquitto is packaged for your distribution — and does not configure any security, so only do this in your own trusted network and switch it off when not needed.

- install the `mosquitto` package
- add a file `/etc/mosquitto/conf.d/listen.conf` with the following content:

```
1 listener 1883
2 allow_anonymous true
```

- start Mosquitto using `systemctl start mosquitto.service`

- (6) *MQTT Tasmota Setup*

Connect to the Tasmota device using a web browser, and open the MQTT configuration page via Configuration > Configure MQTT.

Enter the IP address of the MQTT broker into the “Host” field.

Note down the value shown right of the “Topic” label in parenthesis (typically something like “tasmota_XXXXXX”). This will be needed later on to address the device via MQTT. You can also change the default value to something easier to remember, but this has to be unique if you have multiple devices.

Press Save.

The device will restart and once it is back you should see output in its Console prefixed with “MQT”.

- (7) *Verifying MQTT Communication*

This assumes you have the Mosquitto client tools installed, which are usually available as distribution packages.

You need two terminals to verify MQTT communication works as intended.

```
1 - In terminal 1, run `mosquitto_sub -t 'stat/<topic>/STATUS10'`  
2 - In terminal 2, run `mosquitto_pub -t 'cmd/<topic>/STATUS' -m '10'`
```

Replace `<topic>` with the value noted down in step (6).

Everytime you run the second command, you should see a set of values printed in the first terminal.

- (8) *Continuous Power Measurements*

See scripts in <https://volkerkrause.eu/2020/10/17/kde-cheap-power-measurement-tools.html>.

- (9) *Switching WiFi Networks*

Once connected to a WiFi network, Tasmota will not let you get back to step (2) by default for security reasons, without hard resetting the device (40-second button press). That, however, also removes all settings and the calibration. If you need to move to a different network, there are less drastic options available though, but those can only be taken inside the network you originally connected to:

Under Configuration > Configure WiFi you can add details for a second WiFi access point. Those will be tried alternately with the first configuration by default. This does not compromise security, but requires you to know the details for the network you want to connect to.

You can configure Tasmota to open an access point as in step (2) by default for a minute or so after boot, and then try to connect to the known configurations. This makes booting slower in known networks, and opens the potential for hijacking the device, but it can be convenient when switching to unknown networks. This mode can be enabled in the Console by the command `WifiConfig 2`, and disabled by the command `WifiConfig 4`.

For Tasmota version 11 the 40-second button press reset can leave the device in a non-booting state, whereas resetting from the Console using `Reset 1` doesn't have that problem, but has to be done before disconnecting from the known WiFi as well.

- (10) *Recovering Non-Booting Devices*

First and foremost: **DO NOT CONNECT THE DEVICE TO MAIN POWER!** That would be life-threatening. The entire flashing process is solely powered from 3.3V supplied by the serial adapter. Do not do any of this without having read <https://tasmota.github.io/docs/Getting-Started/>.

With Tasmota 11 you can end up in a non-booting state by merely resetting the device using the 40-second button press. This does not permanently damage the device, and it can be fixed with reflashing via a serial adapter.

The basic process is described in <https://tasmota.github.io/docs/Getting-Started/>. The PCB layout of the Gosund SP 111 can be seen on https://templates.blakadder.com/gosund_SP111_v1_1.

In order for this to work, you need to connect GPIO0 (second pin on bottom left in the above image) to GND **before** powering up (i.e., before connecting with USB). The device LEDs (red and blue) are a useful indicator whether you ended up in the right boot mode: red flashing quickly or both red and blue being on is wrong, while just red being on is correct. Once in that state the connection can be removed (e.g., if you just hold a jumper cable to the pin) and it remains in the right mode until a reboot.

Again: **DO NOT CONNECT THE DEVICE TO MAIN POWER** as this is life-threatening. See above.

(B) Hardware Operating Life

Many FOSS applications run on hardware much older than 5 years, and this category is relatively easily to fulfill for the Blue Angel application. In fact, members of the KDE community have noted that Plasma runs on hardware from even 2005!

To demonstrate compliance for Blue Angel certification, one needs the following information:

- *Reference System Year* — e.g., 2015
- *Model* — e.g., Fujitsu Esprimo 920
- *Processor* — e.g., Intel Core i5-4570
- *Cores* — e.g., 4
- *Clock Speed* — e.g., 3,6 GHz
- *RAM* — e.g., 4 GB
- *Hard Disk (SSD/HDD)* — e.g., 500 GB
- *Graphics Card* — e.g., Intel Ivybridge Desktop
- *Network* — e.g., Realtek Ethernet
- *Cache* — e.g., 6144 KB
- *Mainboard* — e.g., Fujitsu D3171-A1
- *Operating System* — e.g., Ubuntu 18.04

For Blue Angel eco-certification the above data needs to be added to 2 documents called “Annex 1” and “Annex 2”. You can find examples for Okular at the following links:

- Annex 1: <https://invent.kde.org/teams/eco/blue-angel-application/-/blob/master/applications/okular/de-uz-215-eng-annex-1-okular.docx>
- Annex 2: <https://invent.kde.org/teams/eco/blue-angel-application/-/blob/master/applications/okular/de-uz-215-eng-annex-2-okular.xlsx>

(C) User Autonomy

As discussed in Section 2, the Blue Angel user-autonomy criteria cover eight general areas:

1. Data Formats
2. Transparency
3. Continuity Of Support
4. Uninstallability
5. Offline Capability
6. Modularity
7. Freedom From Advertising
8. Documentation

In many FOSS communities, the above user autonomy criteria are a given. Indeed, FOSS projects may take for granted that Free Software respects user autonomy, and in some cases critical information from the above list is missing from websites, manuals, wikis, etc. Things like support for open standards in data formats, uninstallability, or continuity of support.

Documenting this information is important, both for fulfilling the Blue Angel award criteria and for giving users information about long-term sustainable use of their software and thus hardware. KDE and FOSS communities know Free Software respects its users — now let's make sure everyone else knows it too!

This is not an exhaustive presentation for each of the above categories of the Blue Angel criteria. Rather, aspects of the criteria which KDE/FOSS projects can easily fill out in their documentation is the focus, which is already most of the work. For the full criteria, see Section 3.1.3 in the award criteria.

2.1 Data Formats

The main information to include in documentation:

- Which (open) data formats are supported — with links to specifications, e.g., PDF?
- Also of interest: Are there examples of other software products that process these data formats?

For an example of the online documentation of supported data formats for Okular, see <https://okular.kde.org/formats/>.

For an example of documentation in a Blue Angel application, see <https://invent.kde.org/teams/eco/blue-angel-application/-/blob/master/applications/okular/de-uz-215-eng-annex-4-okular.md>.

2.2 Transparency Of The Software Product

When missing, provide links to documentation of the API, source code, and the license.

For the sake of example, for KMail:

- KDE PIM API documentation: <https://api.kde.org/kdepim/index.html>
- Source code: <https://invent.kde.org/pim/kmail>
- License: <https://invent.kde.org/pim/kmail/-/blob/master/LICENSES>

For an example of documentation in a Blauer Engel application, see: <https://invent.kde.org/teams/eco/blue-angel-application/-/blob/master/applications/kmail/de-uz-215-eng-annex-5-kmail.md>.

2.3 Continuity Of Support

Examples of details about continuity of support to add to documentation:

- Information about how long the software has been supported for (with links to release announcements)
- Release schedule and details (e.g., who maintains the software)
- Statement that updates are free of charge
- Declaration on how the free and open source software license enables continuous support indefinitely
- Information about whether, and how, functional and security updates may be installed separately

An example of Okular's continuity of support documentation for Blauer Engel certification can be found in Section 3.1.3.3 here: <https://invent.kde.org/teams/eco/blue-angel-application/-/blob/master/applications/okular/de-uz-215-eng-annex-6-okular.md>.

2.4 Uninstallability

How can one completely uninstall the software? Relevant details may include:

- Uninstallation depends on how the software was installed (source code or binary)
- Examples of uninstallation instructions (source code or package managers, with relevant links to documentation)
- Information about whether user-generated data is also removed when uninstalling a program

An example of Okular's uninstallability documentation for Blauer Engel certification can be found in Section 3.1.3.4 here: <https://invent.kde.org/teams/eco/blue-angel-application/-/blob/master/applications/okular/de-uz-215-eng-annex-6-okular.md>.

2.5 Offline Capability

This could be one of the easiest areas to document (alongside freedom from advertising; see below): Does the software require external connections such as a license server in order to run? If not, no network connection is needed and the software is capable of offline usage, and this should be documented.

An example of Okular's offline capability documentation for Blauer Engel certification can be found in Section 3.1.3.5 here: <https://invent.kde.org/teams/eco/blue-angel-application/-/blob/master/applications/okular/de-uz-215-eng-annex-6-okular.md>.

2.6 Modularity

Information to document includes:

- What aspects of the software are modular and can be deactivated during installation?
- For instance, can the software manuals or translations be installed separately?
- Are any modules unrelated to the core functionality of the software included with installation, such as tracking modules or cloud integration? If not, document it!

An example of Okular's modularity documentation for Blauer Engel certification can be found in Section 3.1.3.6 here: <https://invent.kde.org/teams/eco/blue-angel-application/-/blob/master/applications/okular/de-uz-215-eng-annex-6-okular.md>.

2.7 Freedom From Advertising

If the software does not display advertising, make this explicit in manuals and wikis and declare it in the Blue Angel application document.

2.8 Documentation

This includes the following:

- General process for installing/uninstalling the software? May include generic instructions or tutorials for a specific desktop environment or package manager.
- Data import/export process?
- What can users do to reduce the use of resources (e.g., configuration options for improving performance)?
- Does the software have any resource-intensive functionality not necessary for the core functionality? If not, let's tell people!

- Licensing terms related to further development of the software products, with links to source code and license?
- Who supports the development of the software?
- Does the software collect any personal data? Is it compliant with existing data protection laws? If yes, document it!
- What is the privacy policy? Is there telemetry, and if yes, how does the software handle data security, data collection, and data transmission? Also, are there ads or tracking embedded in the software? If not, excellent, now let everyone else know!

An example of Okular's product documentation for Blauer Engel certification can be found in Section 3.1.3.8 here: <https://invent.kde.org/teams/eco/blue-angel-application/-/blob/master/applications/okular/de-uz-215-eng-annex-6-okular.md>.

Examples Below are examples of Blauer Engel documentation for Okular.

- Annex 4: Data Formats (Section 2.1)
 - <https://invent.kde.org/teams/eco/blue-angel-application/-/blob/master/applications/okular/de-uz-215-eng-annex-4-okular.md>
- Annex 5: Open Standards (Transparency Section 2.2)
 - <https://invent.kde.org/teams/eco/blue-angel-application/-/blob/master/applications/okular/de-uz-215-eng-annex-5-okular.md>
- Annex 6: Product Information (Sections 2.3–2.8)
 - <https://invent.kde.org/teams/eco/blue-angel-application/-/blob/master/applications/okular/de-uz-215-eng-annex-6-okular.md>
- See also Annex 1
 - <https://invent.kde.org/teams/eco/blue-angel-application/-/blob/master/applications/okular/de-uz-215-eng-annex-1-okular.docx> == TODO ==